SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

NAME :Y.jaswith

REG NO :11249A419

CLASS : S4

SUBJECT CODE : BCSD244A12

SUBJECT NAME :DATA SCIENCE TOOLKIT LAB

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF COMPUTER

SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to cer fy that this is the bonafide record of work done by

Mr/Ms. _____Y.jaswith_____with

Reg. No 11249A419 done by of II-B.E.(CSE) in –

DATA SCIENCE TOOLKIT LAB for Lab during the academic year 2025 2026.

Sta on : ENATHUR Date :

Staff-in-charge Head of the

Department

Submi ed for the prac cal examina on held on _____

Examine-1
Examine -2

INDEX

SI.NO	PROGRAM NAME	DATE	PAGE NUM	SIGNATURE
01(1.1)	INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE	02-02-2026	1	
1.2	PANDAS	02-02-2026	5	
1.3	MATPLOT LIB	02-02-2026	7	
1.4	SCIKIT LEARN	09-02-2026	9	
02	PERFORM DATA EXPLORATION AND PRE PROCESSING IN PYTHON	14-02-2026	11	
03	IMPLEMENT REGULARIZED LINEAR REGRESSION	14-02-2026	13	
04	IMPLEMENT NAVIE BAYES CLASSIFIER	23-02-2026	14	
05	IMPLEMENT REGULARIZED LOGISTIC REGRESSION	02-03-2026	15	
06	BUILD MODULES USING DIFFERENT ENSEMBLING TECHNIQUES	09-03-2026	17	

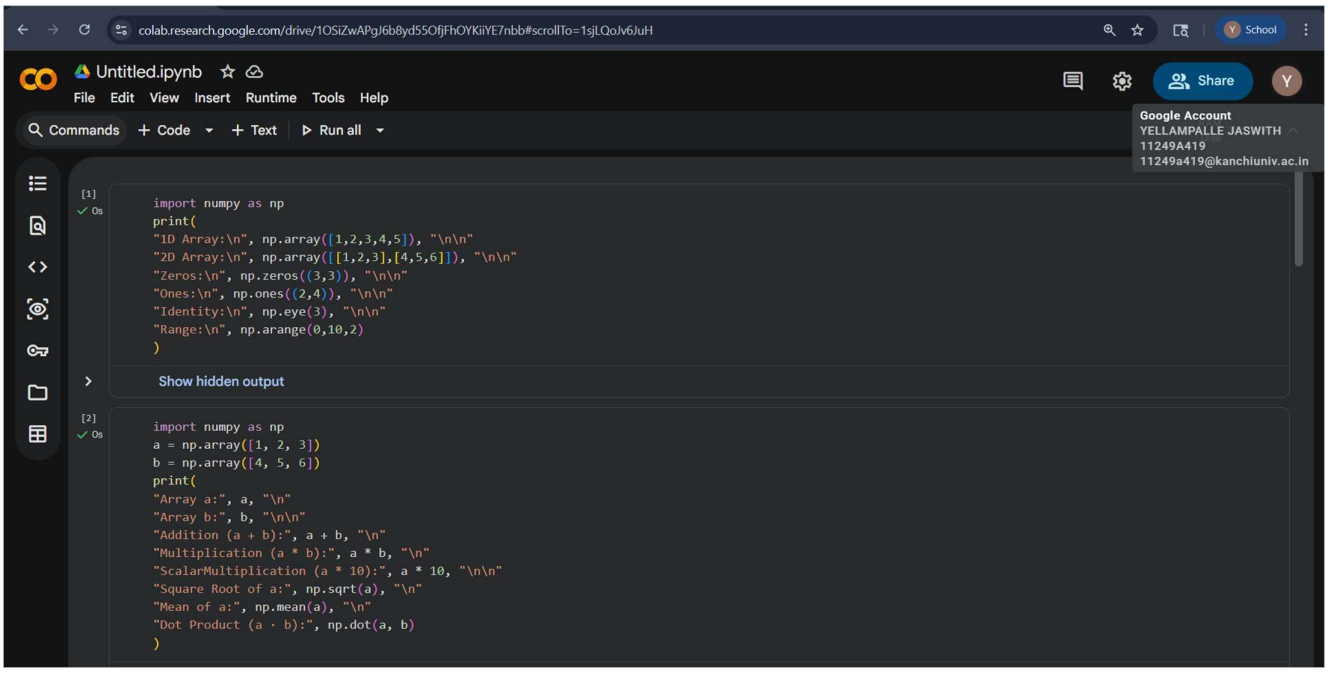
07	BUILD MODELS USING DECESION TREES	09-03-2026	18	
08	BUILD THE MODELS USING DIFFERENT KERNAL FUNCTIONS	16-03-2026	19	
09	IMPLEMENT K-NEAREST NEIGHBOURS (KNN)CLASSIFICATION	30-03-2026	20	
10	K-MEAN CLUSTERING WITH PCA AND ELBOW METHOD	30-03-2026	21	

INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE

Exp-01(1.1) Date-02-02-2026

Aim:

To learn basic Python data science libraries like NumPy and Pandas.



colab.research.google.com/drive/1OSiZwAPgJ6b8yd55OfjFhOYKiiYE7nbb#scrollTo=1sjLQoJv6JuH

Untitled.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
YELLAMPALLE JASWITH
11249A419
11249a419@kanchiuniv.ac.in

[3]

```python
import numpy as np
matrix = np.array([[10, 20, 30],
[40, 50, 60],
[70, 80, 90]])
print(
"Matrix:\n",matrix, "\n\n"
"Element at [1,2]:", matrix[1, 2], "\n\n"
"First Row:",matrix[0, :], "\n"
"Second Column:",matrix[:, 1], "\n\n"
"Top-left 2x2 Sub-matrix:\n",matrix[0:2, 0:2]
)
```

Show hidden output

[4]

```python
import numpy as np

# 1. Array Creation
a = np.array([1, 2, 3, 4, 5])
b = np.arange(1, 10, 2)
c = np.linspace(0, 1, 5)
d = np.zeros((2, 3))
e = np.ones((2, 2))
f = np.eye(3)
g = np.random.randint(1, 100, size=(3, 3))

print("Array a:", a)
print("Arange b:", b)
print("Linspace c:", c)
print("Zeros d:\n", d)
print("Ones e:\n", e)
```

colab.research.google.com/drive/1OSiZwAPgJ6b8yd55OfjFhOYKiiYE7nbb#scrollTo=1sjLQoJv6JuH

Untitled.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
YELLAMPALLE JASWITH 11249A419
11249a419@kanchiuniv.ac.in

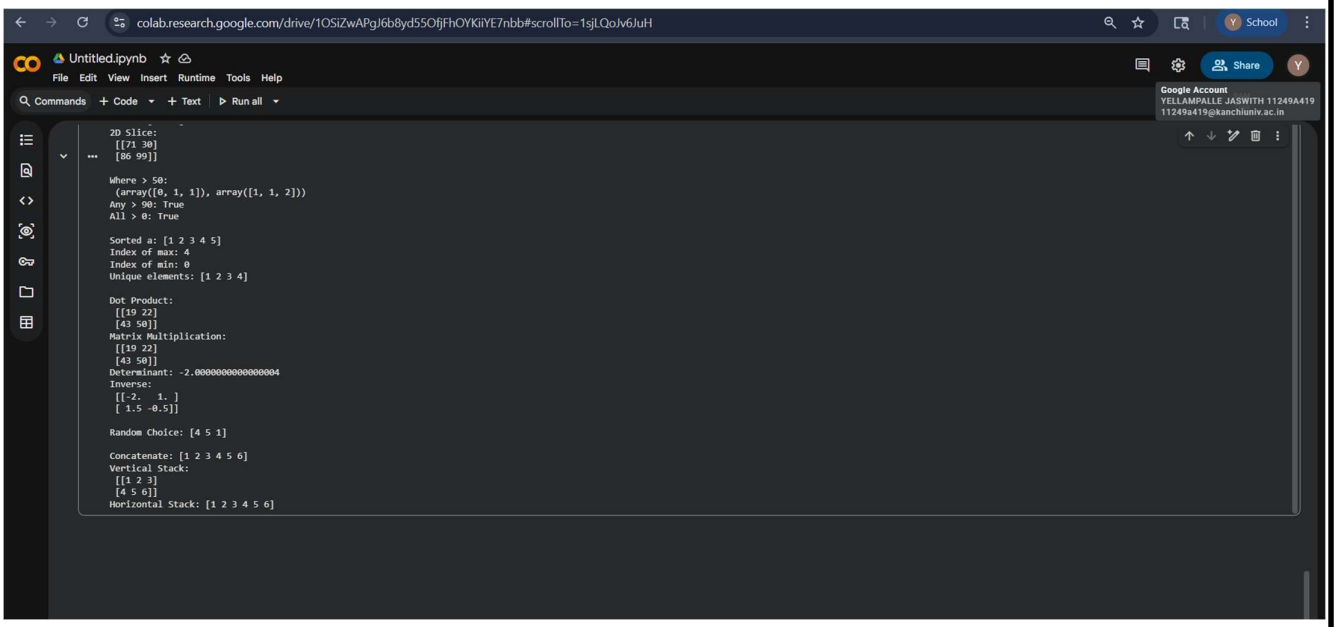
[4]

```python
# 2. Array Properties
print("\nShape:", g.shape)
print("Dimensions:", g.ndim)
print("Size:", g.size)
print("Data type:", g.dtype)

# 3. Reshape & Transpose
reshaped = g.reshape(1, 9)
print("\nReshaped array:", reshaped)
print("Transpose:\n", g.T)
print("Flattened:", g.flatten())

# 4. Mathematical Operations
print("\nAdd:", np.add(a, 2))
print("Multiply:", np.multiply(a, 3))
print("Square Root:", np.sqrt(a))
print("Power:", np.power(a, 2))
print("Absolute:", np.abs([-1, -2, 3]))

# 5. Statistical Functions
print("\nMean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Sum:", np.sum(a))
print("Percentile (50):", np.percentile(a, 50))

# 6. Axis-based operations
print("\nColumn-wise Sum:\n", np.sum(g, axis=0))
print("Row-wise Mean:\n", np.mean(g, axis=1))

# 7. Indexing & Slicing
print("\nFirst element:", a[0])
print("Slice:", a[1:4])
print("2D Slice:\n", g[0:2, 1:3])

# 8. Boolean & Conditional
print("\nWhere > 50:\n", np.where(g > 50))
print("Any > 90:", np.any(g > 90))
print("All > 0:", np.all(g > 0))

# 9. Sorting & Searching
print("\nSorted a:", np.sort(a))
```

```
y = np.array([[5, 6], [7, 8]])

print("\nDot Product:\n", np.dot(x, y))
print("Matrix Multiplication:\n", np.matmul(x, y))
print("Determinant:", np.linalg.det(x))
print("Inverse:\n", np.linalg.inv(x))

# 11. Random Functions
np.random.seed(1)
rand_sample = np.random.choice(a, size=3)
print("\nRandom Choice:", rand_sample)

# 12. Stack & Concatenate
h1 = np.array([1, 2, 3])
h2 = np.array([4, 5, 6])

print("\nConcatenate:", np.concatenate((h1, h2)))
print("Vertical Stack:\n", np.vstack((h1, h2)))
print("Horizontal Stack:", np.hstack((h1, h2)))
```

OUTPUT-

```
Array a: [1 2 3 4 5]
Arange b: [1 3 5 7 9]
Linspace c: [0.   0.25 0.5  0.75 1.  ]
Zeros d:
 [[0. 0. 0.]
 [0. 0. 0.]]
Ones e:
 [[1. 1.]
 [1. 1.]]
Identity f:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random g:
 [[12 71 30]
 [ 5 86 99]
 [46 35 43]]

Shape: (3, 3)
Dimensions: 2
Size: 9
Data type: int64

Reshaped array: [[12 71 30  5 86 99 46 35 43]]
Transpose:
 [[12  5 46]
 [71 86 35]
 [30 99 43]]
Flattened: [12 71 30  5 86 99 46 35 43]

Add: [3 4 5 6 7]
Multiply: [ 3  6  9 12 15]
Square Root: [1.         1.41421356 1.73205081 2.         2.23606798]
Power: [ 1  4  9 16 25]
Absolute: [1 2 3]

Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Minimum: 1
Maximum: 5
Sum: 15
Percentile (50): 3.0

Column-wise Sum:
 [ 63 192 172]
Row-wise Mean:
```



Result:

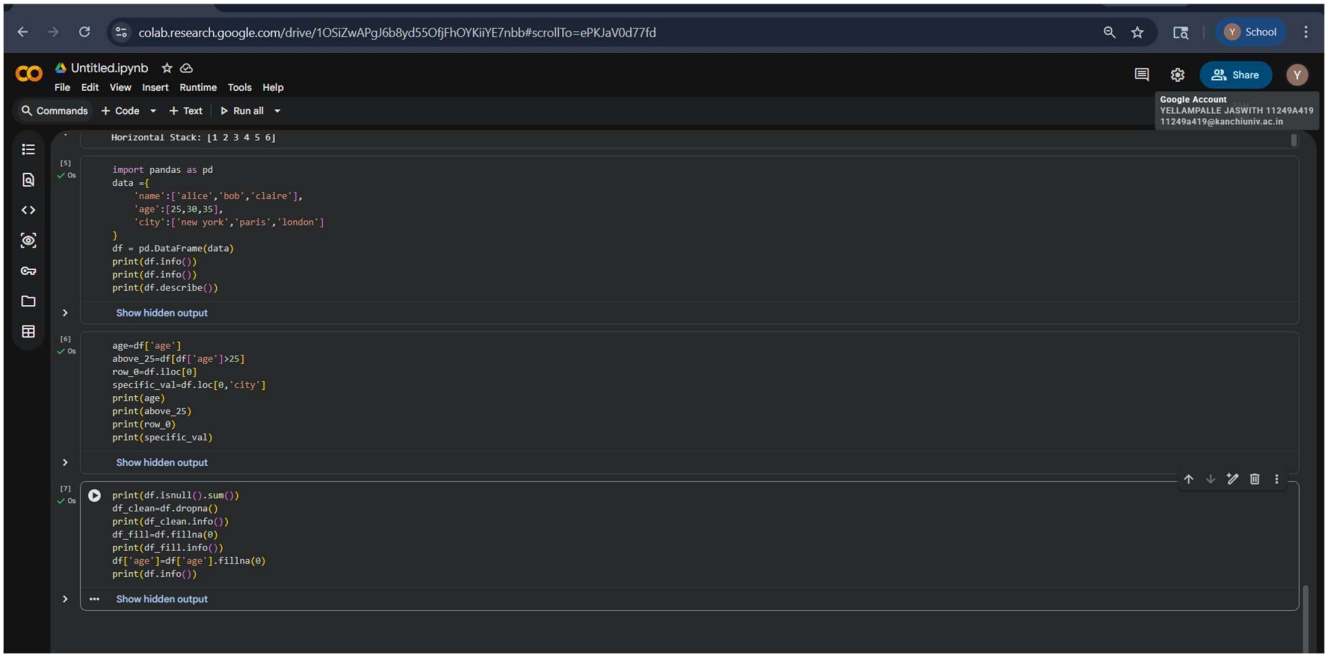
Thus, it is implemented Introduc on to Python Libraries for Data Science successfully.

PANDAS

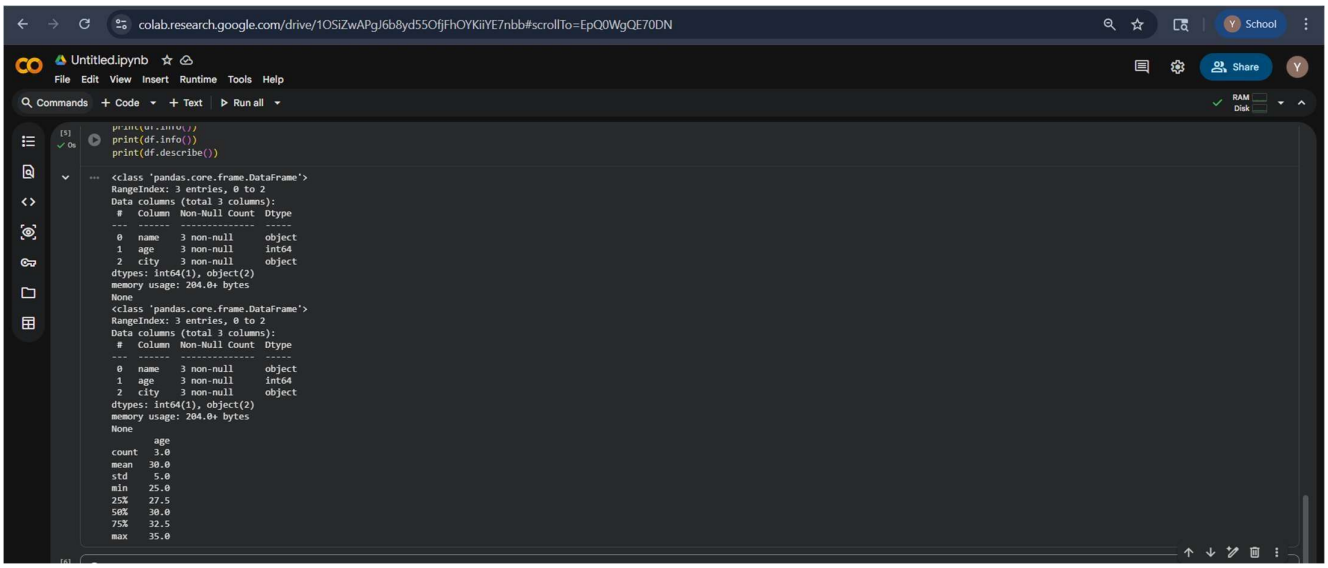
EXP-1.2 DATE-02-02-2026

Aim:

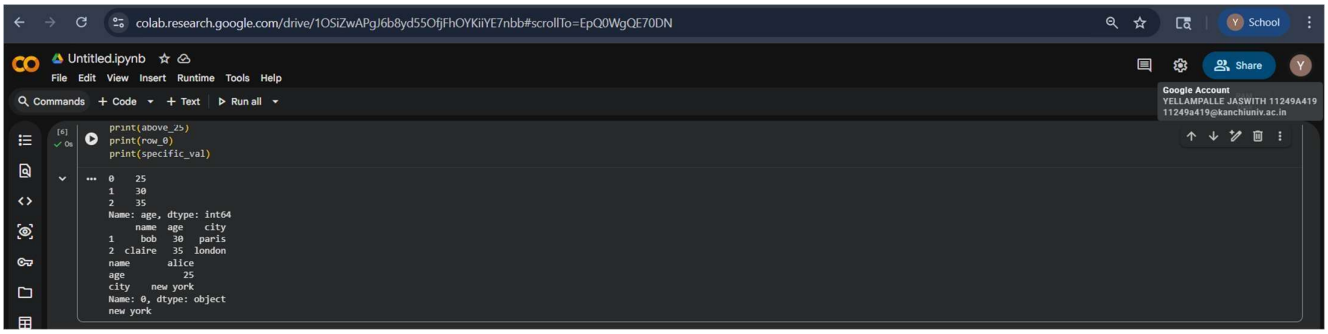
To study the Pandas library for data handling and analysis.



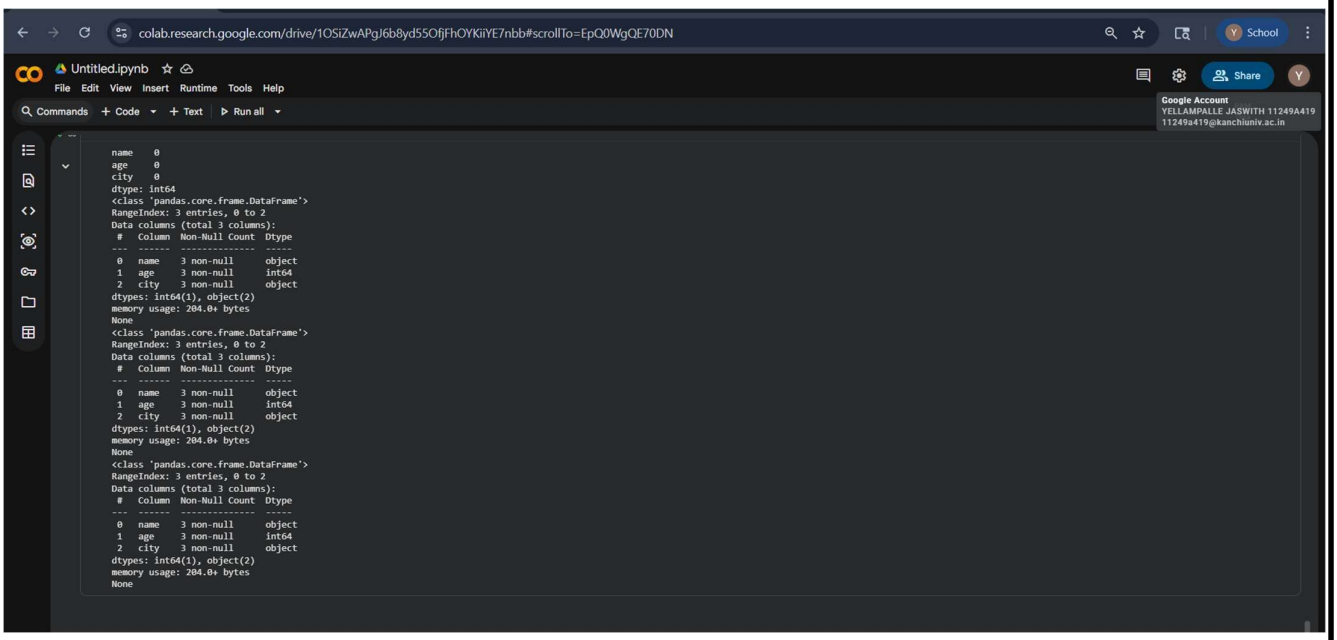
Output:



Output:



OUTPUT-



Result:

Thus, it is implemented Pandas successfully.

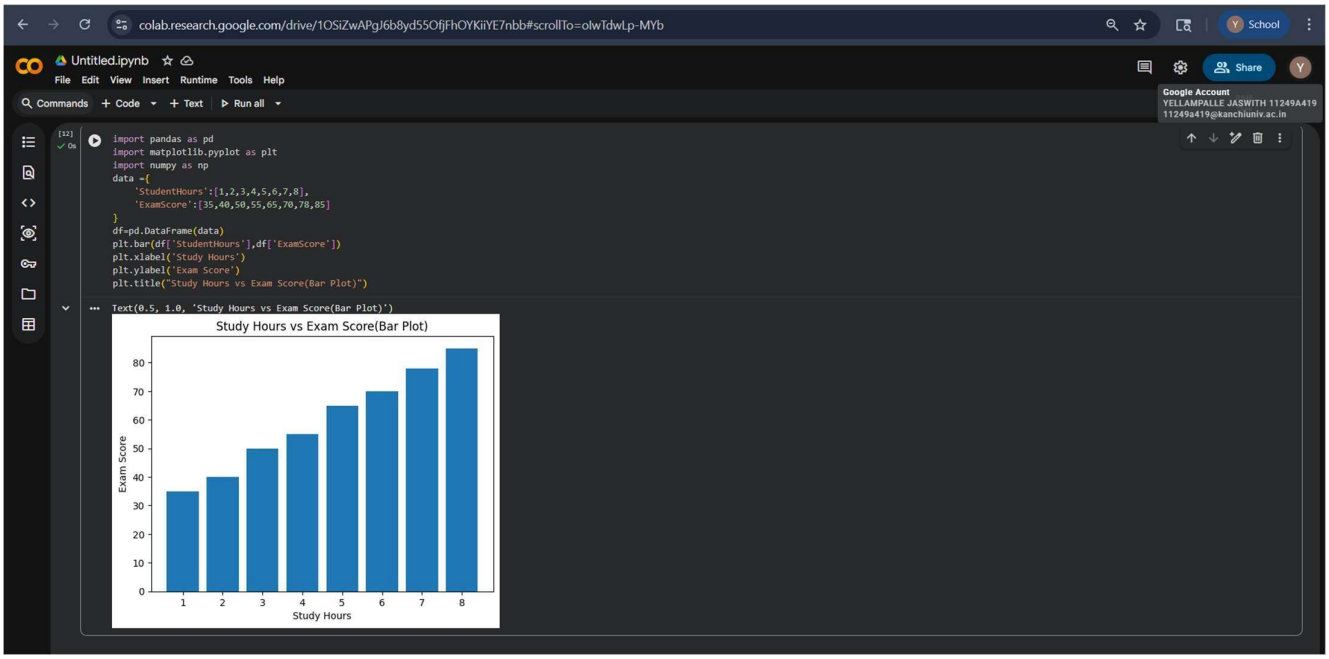
MATPLOT LIB

EXP-1.3 DATE-02-02-2026

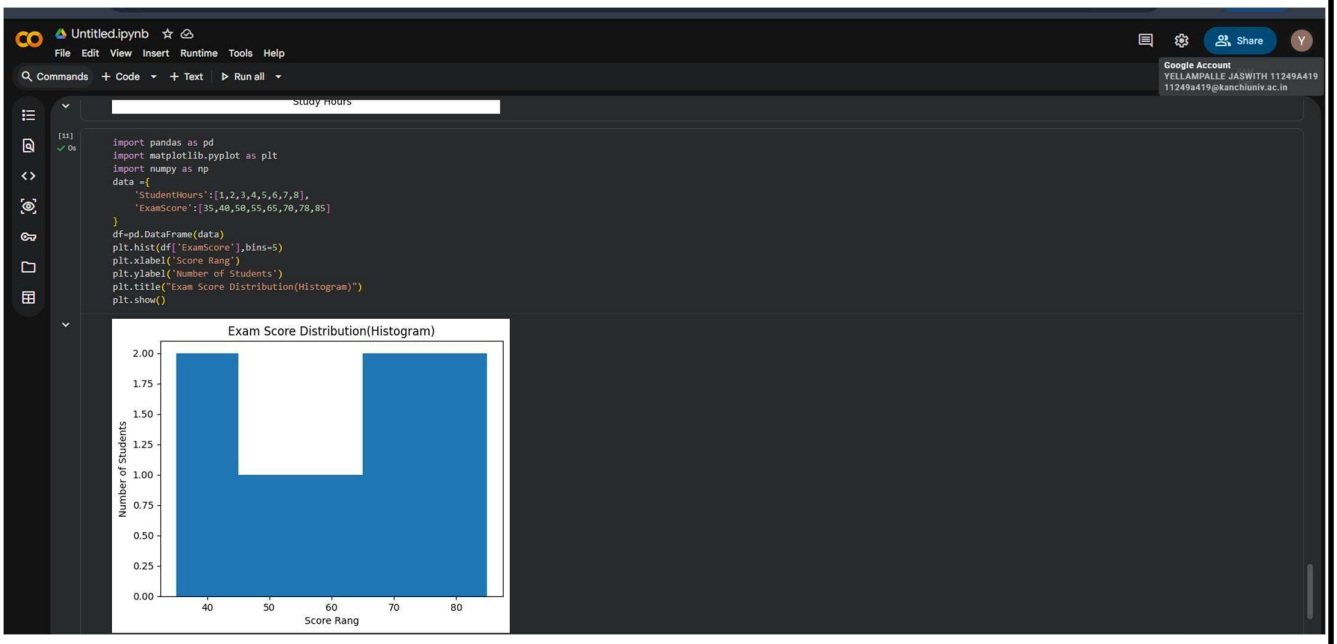
Aim:

To study the Matplotlib library for data visualiza on.

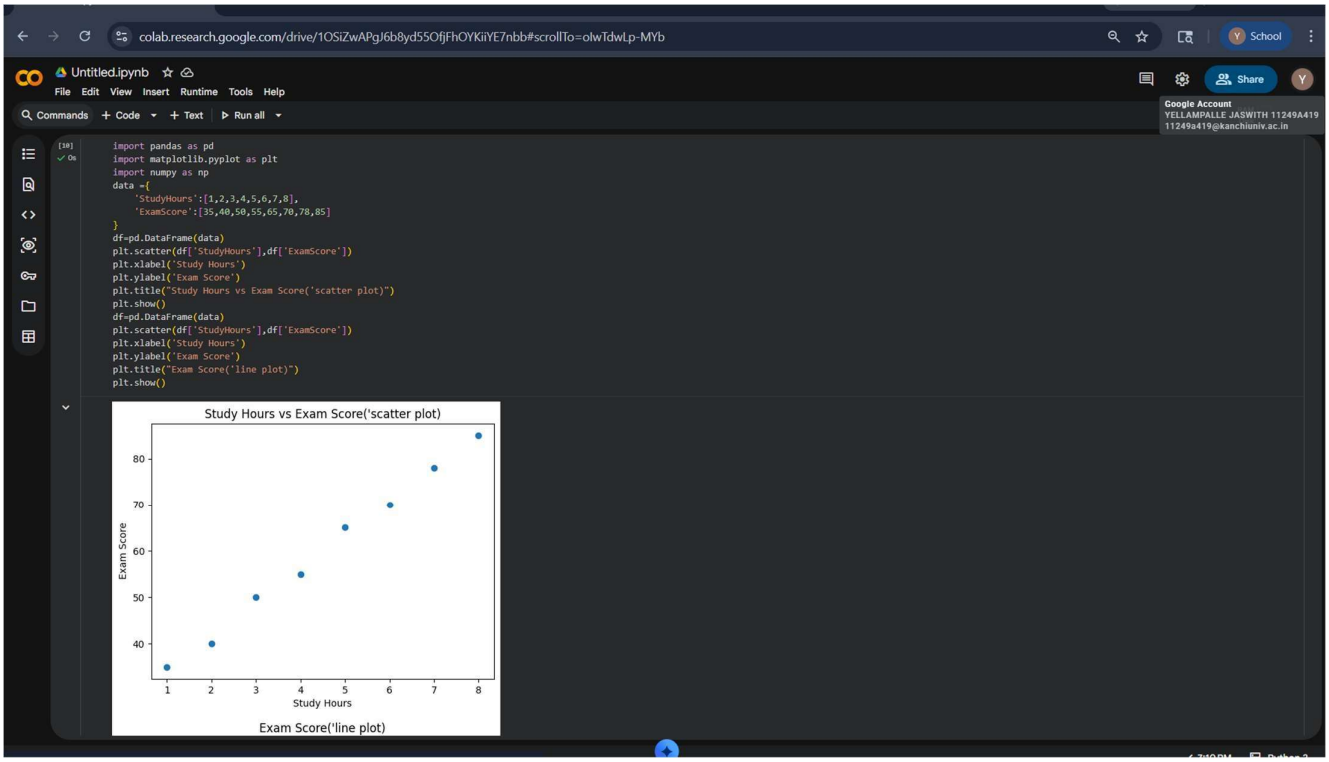
Program with output::



PROGRAM WITH OUTPUT-



PROGRAM WITH OUTPUT-



Result:

Thus, it is implemented Matplotlib successfully.

SCIKIT LEARN

EXP N-1.4 DATE-09-02-2026

Aim:

To study the scikit-learn library for machine learning.

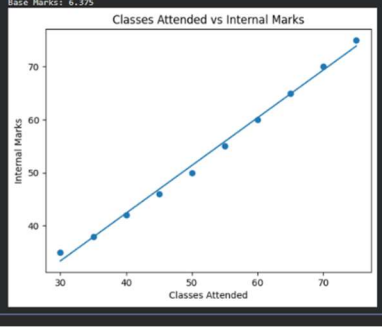
Program:

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
#  CREATE DATASET
data = {

    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]

}

df = pd.DataFrame(data)
print(df)
#  FEATURES & TARGET
X = df[['Classes_Attended']]
y = df['Internal_Marks']

#  TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)
#  TRAIN MODEL

model = LinearRegression()
model.fit(X_train, y_train)

#  PREDICTION

predictions = model.predict(X_test)
#  EVALUATION
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)

#  VISUALIZATION

plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

OUTPUT-

```python
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

```
   Student  Classes_Attended  Internal_Marks
0       S1                30              35
1       S2                35              38
2       S3                40              42
3       S4                45              46
4       S5                50              50
5       S6                55              55
6       S7                60              60
7       S8                65              65
8       S9                70              70
9      S10                75              75
MSE: 0.578125
Marks per Class: 0.9
Base Marks: 6.375
```



Result:

Thus, it is implemented Scikit-learn successfully.

PERFORM DATA EXPLORATION AND PREPROCESSING IN PYTHON

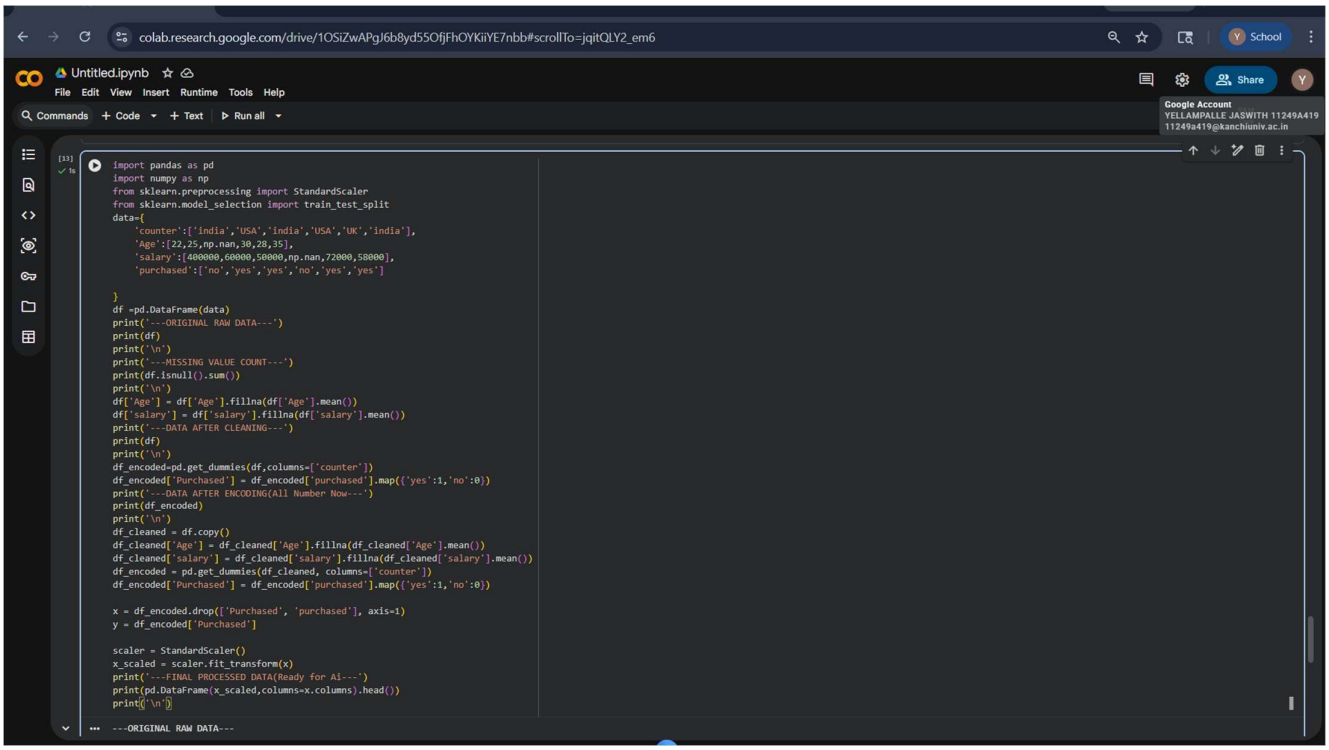
EXP-02

DATE-14-02-2026

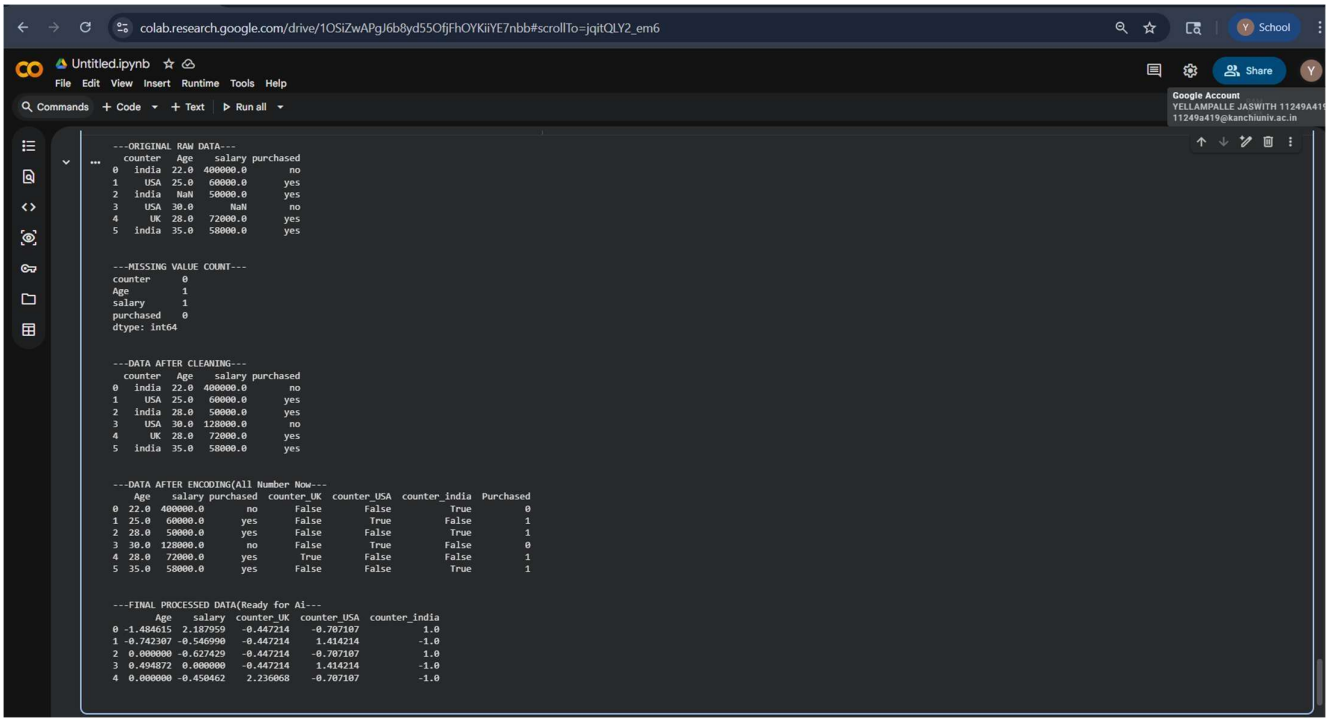
Aim:

To perform data explora on and preprocessing using Python.

PROGRAM:



Out put:



```
---ORIGINAL RAW DATA---
  counter   Age    salary purchased
0   india  22.0  400000.0        no
1     USA  25.0   60000.0       yes
2   india   NaN   50000.0       yes
3     USA  30.0       NaN        no
4      UK  28.0   72000.0       yes
5   india  35.0   58000.0       yes


---MISSING VALUE COUNT---
counter      0
Age          1
salary       1
purchased    0
dtype: int64


---DATA AFTER CLEANING---
  counter   Age    salary purchased
0   india  22.0  400000.0        no
1     USA  25.0   60000.0       yes
2   india  28.0   50000.0       yes
3     USA  30.0  128000.0        no
4      UK  28.0   72000.0       yes
5   india  35.0   58000.0       yes


---DATA AFTER ENCODING(All Number Now---
    Age    salary purchased  counter_UK  counter_USA  counter_india  Purchased
0  22.0  400000.0        no       False        False           True          0
1  25.0   60000.0       yes       False         True          False          1
2  28.0   50000.0       yes       False        False           True          1
3  30.0  128000.0        no       False         True          False          0
4  28.0   72000.0       yes        True        False          False          1
5  35.0   58000.0       yes       False        False           True          1


---FINAL PROCESSED DATA(Ready for Ai---
        Age    salary  counter_UK  counter_USA  counter_india
0 -1.484615  2.187959   -0.447214    -0.707107            1.0
1 -0.742307 -0.546990   -0.447214     1.414214           -1.0
2  0.000000 -0.627429   -0.447214    -0.707107            1.0
3  0.494872  0.000000   -0.447214     1.414214           -1.0
4  0.000000 -0.450462    2.236068    -0.707107           -1.0
```

Result:

The dataset was explored, cleaned, and pre processed successfully, making it ready for further analysis or model building.

IMPLEMENT REGULARIZED LINEAR REGRESSION

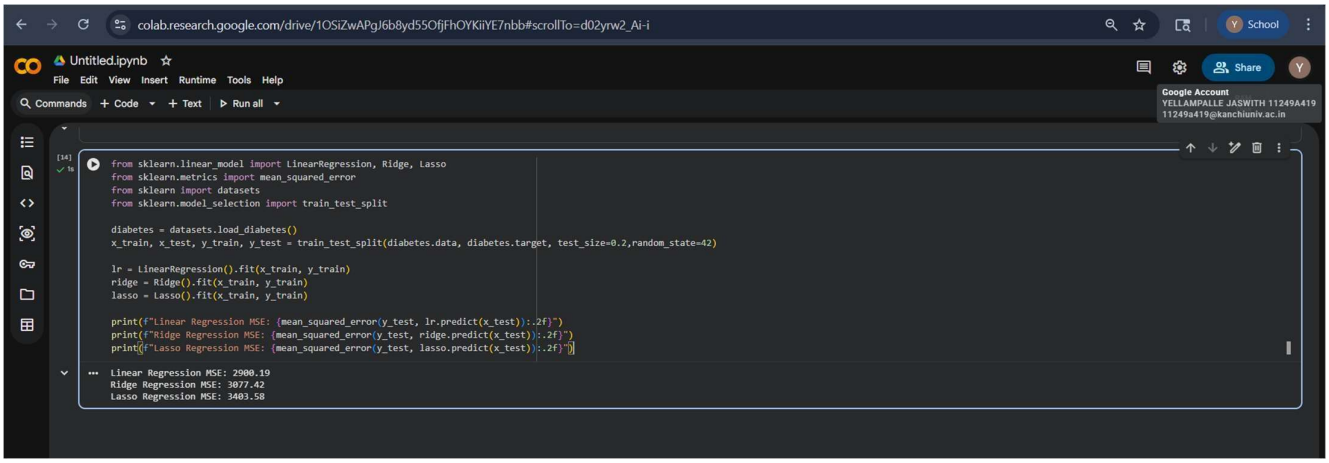
EXP-03

DATE-14-02-2026

Aim

To implement Regularized Linear Regression using Python.

Prpgram with outpt :



Result

The regularized linear regression model was successfully implemented using Python. The model reduced overfi ng and improved predic on performance by applying regulariza on techniques such as Ridge or Lasso regression.

IMPLEMENT NAVIE BAYES CLASSIFIER

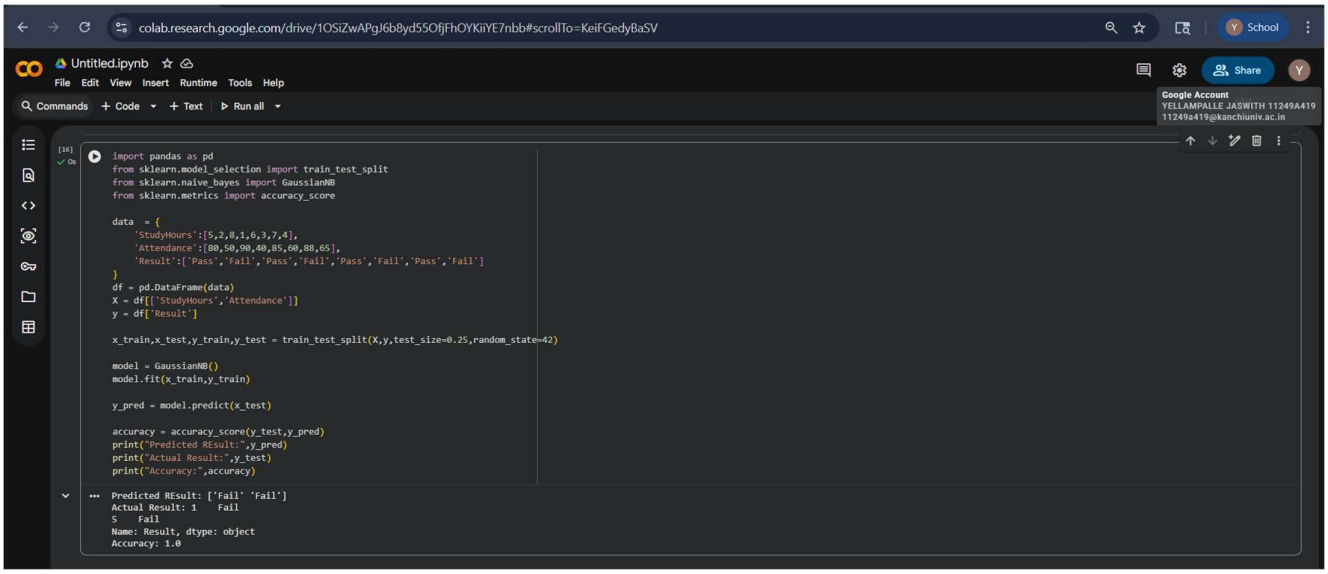
EXP-04

DATE-23-02-2026

Aim

To implement a Naive Bayes classifier using Python for classifica on of data.

PROGRAM WITH OUTPUT



Result

The Naive Bayes classifier was successfully implemented using Python. The model was trained and used to classify the data effec vely, providing accurate predic ons based on probability.

IMPLEMENT REGULARIZED LOGISTIC REGRESSION

EXP-05

DATE-02-03-2026

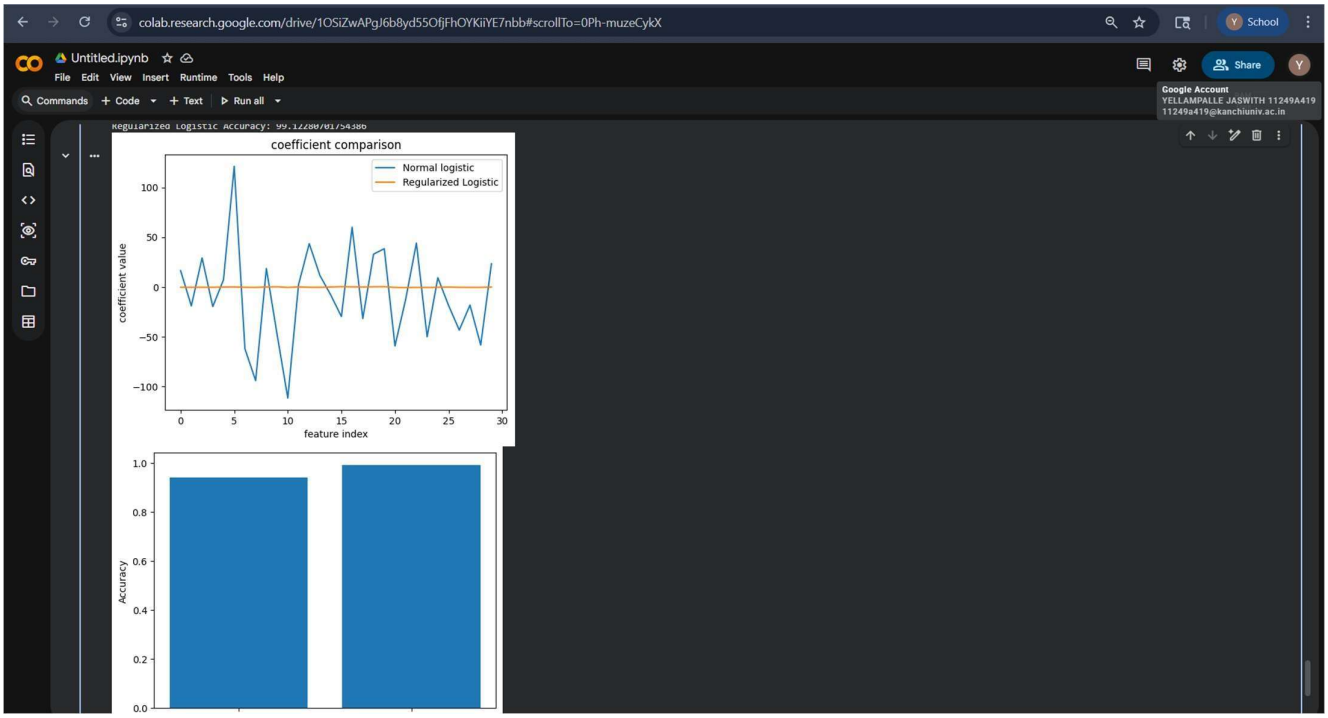
Aim:

To implement Regularized Logis c Regression using Python.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,random_state=42)

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(x_train_scaled, y_train)
normal_pred = normal_model.predict(x_test_scaled)
normal_accuracy = accuracy_score(y_test, normal_pred)

regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(x_train_scaled, y_train)
regularized_pred = regularized_model.predict(x_test_scaled)
regularized_accuracy = accuracy_score(y_test, regularized_pred)

print("Normal Logistic Accuracy:", normal_accuracy *100)
print("Regularized Logistic Accuracy:", regularized_accuracy * 100)

plt.figure()
plt.plot(normal_model.coef_.flatten(), label='Normal logistic')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("coefficient comparison")
plt.xlabel("feature index")
plt.ylabel("coefficient value")
plt.show()

plt.figure()
models = ['Normal logistic', 'Regularized logistic']
accuracies = [normal_accuracy, regularized_accuracy]
plt.bar(models, accuracies)
plt.xlabel('Accuracy Comparison')
```

OUTPUT-



Result

The regularized logis c regression model was successfully implemented using Python. Regulariza on helped reduce overfi ng and improved the model's classifica on performance and generaliza on on the dataset.

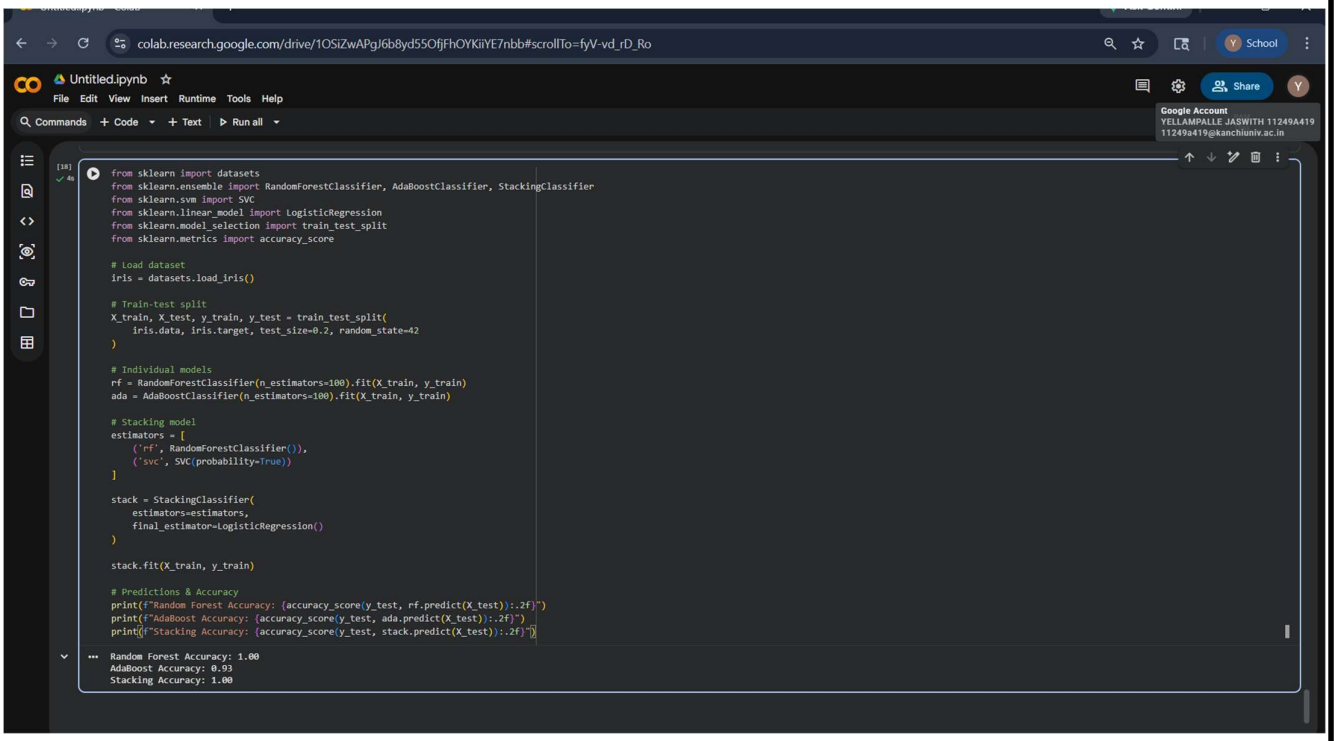
BUILD MODULES USING DIFFERENT ENSEMBLING TECHNIQUES

EXP-06

DATE-09-03-2026

Aim:

PROGRAM WITH OUTPUT



Result

The ensemble models were successfully built using techniques such as Bagging, Boos ng, and Random Forest. These methods improved predic on accuracy and reduced overfi ng compared to individual models.

BUILD MODELS USING DECESION TREES

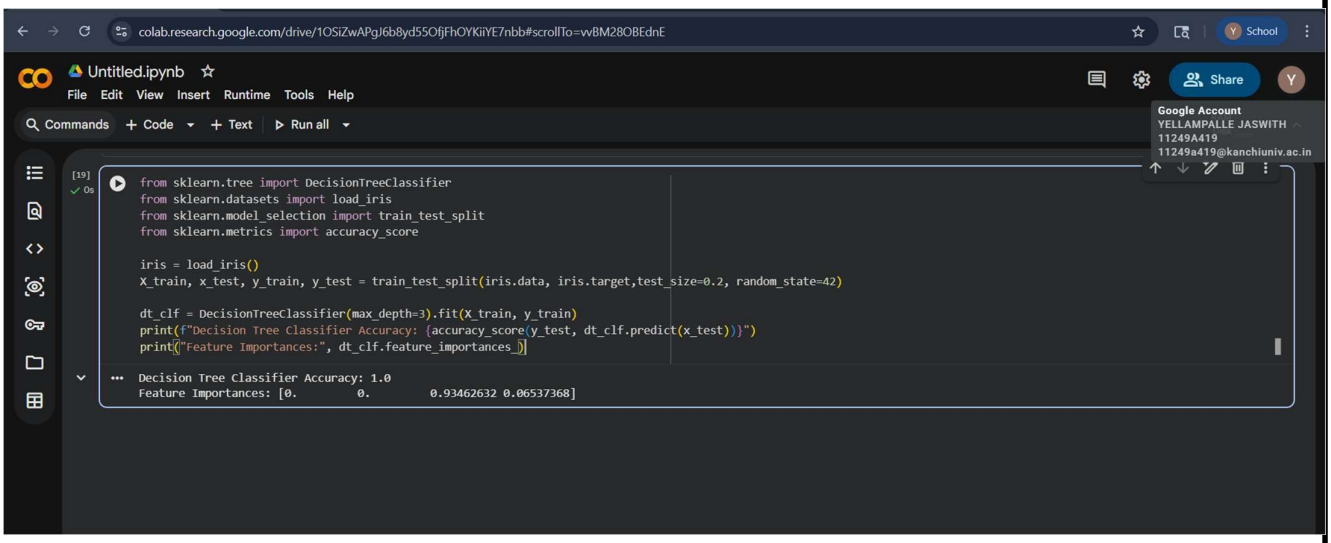
EXP-7

DATE-09-03-2026

Aim:

To build a classifica on model using Decision Trees in Python.

PROGRAM WITH OUTPUT



Result:

The Decision Tree model was successfully implemented using Python. The model learned decision rules from the dataset and provided accurate predic ons for classifica on tasks.

BUILD THE MODELS USING DIFFERENT KERNAL FUNCTIONS

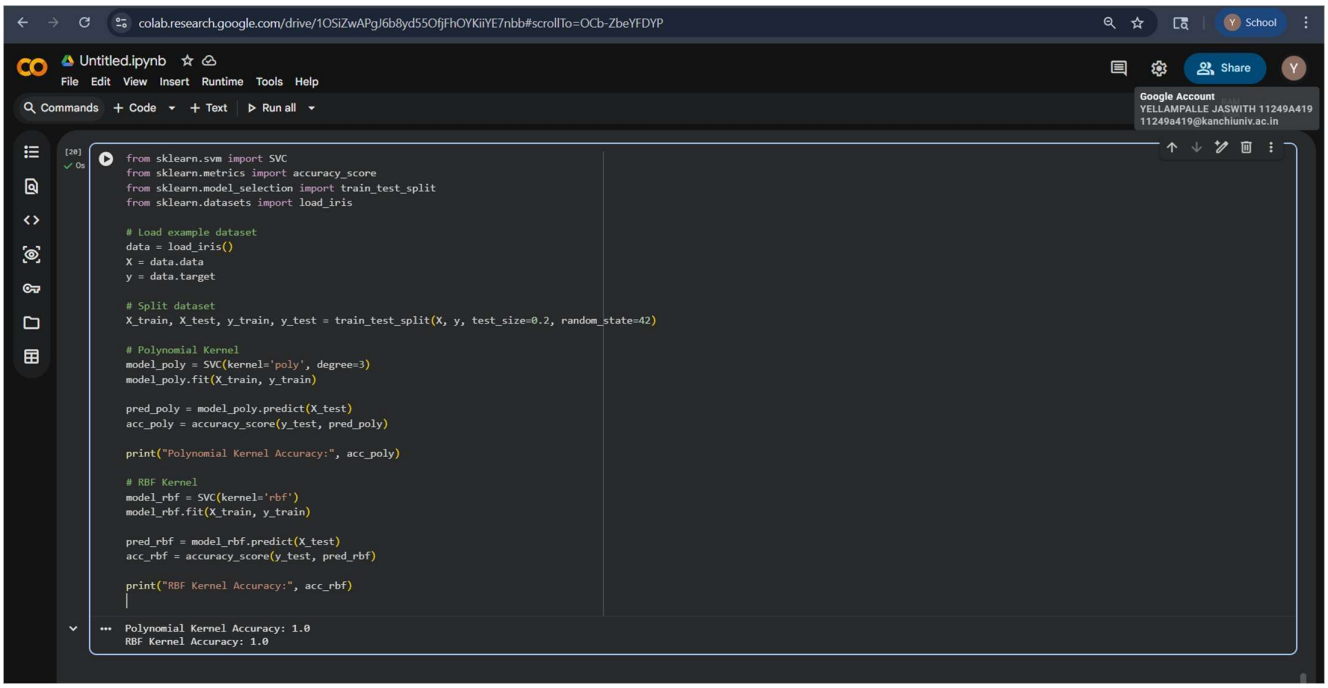
EXP-08

DATE-16-03-2026

Aim

To build models using different kernel func ons in Python.

PROGRAM WITH OUTPUT:



```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

# Load example dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)

pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)

print("Polynomial Kernel Accuracy:", acc_poly)

# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)

pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)

print("RBF Kernel Accuracy:", acc_rbf)
```

```
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0
```

Result

The model was successfully implemented using kernel-based methods (such as SVM). Different kernel func ons like linear, polynomial, and RBF were used, and they improved classifica on performance based on the dataset structure.

IMPLEMENT K-NEAREST NEIGHBOURS

(KNN)CLASSIFICATION EXP-09 DATE-30-32026

Aim:

To implement the K-Nearest Neighbours (KNN) classifica on algorithm using Python.

PROGRAM:

```python
# Step 1: Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Create a small example dataset (similar to mobile dataset)
data = {
    'battery_power': [842, 1021, 563, 615, 1821, 1859, 1954, 1445],
    'ram': [2549, 2631, 2603, 2769, 1411, 1067, 275, 296],
    'px_height': [20, 905, 1263, 1216, 1208, 1004, 381, 512],
    'px_width': [756, 1988, 1716, 1786, 1212, 1654, 1018, 1140],
    'price_range': [1, 2, 2, 2, 1, 1, 0, 0]  # Target (0-3 in real dataset)
}

df = pd.DataFrame(data)

# Step 3: Features and target
X = df.drop(columns=['price_range'])
y = df['price_range']

# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=42, stratify=y
)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=3)

# Step 7: Train model
knn.fit(X_train, y_train)

# Step 8: Prediction
y_pred = knn.predict(X_test)

# Step 9: Evaluation
print("Actual:", list(y_test))
print("Predicted:", list(y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
Actual: [2, 0, 1, 1]
Predicted: [np.int64(2), np.int64(0), np.int64(0), np.int64(0)]

Accuracy: 0.5

Confusion Matrix:
 [[1 0 0]
 [2 0 0]
 [0 0 1]]

Classification Report:
              precision    recall  f1-score   support

           0       0.33      1.00      0.50         1
           1       0.00      0.00      0.00         2
           2       1.00      1.00      1.00         1

    accuracy                           0.50         4
   macro avg       0.44      0.67      0.50         4
weighted avg       0.33      0.50      0.38         4

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavio
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavio
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavio
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Result

The KNN classifica on model was successfully implemented using Python. The model classified data based on the nearest neighbors and produced effec ve and accurate predic ons for the given dataset.

K-MEAN CLUSTERING WITH PCA AND ELBOW METHOD

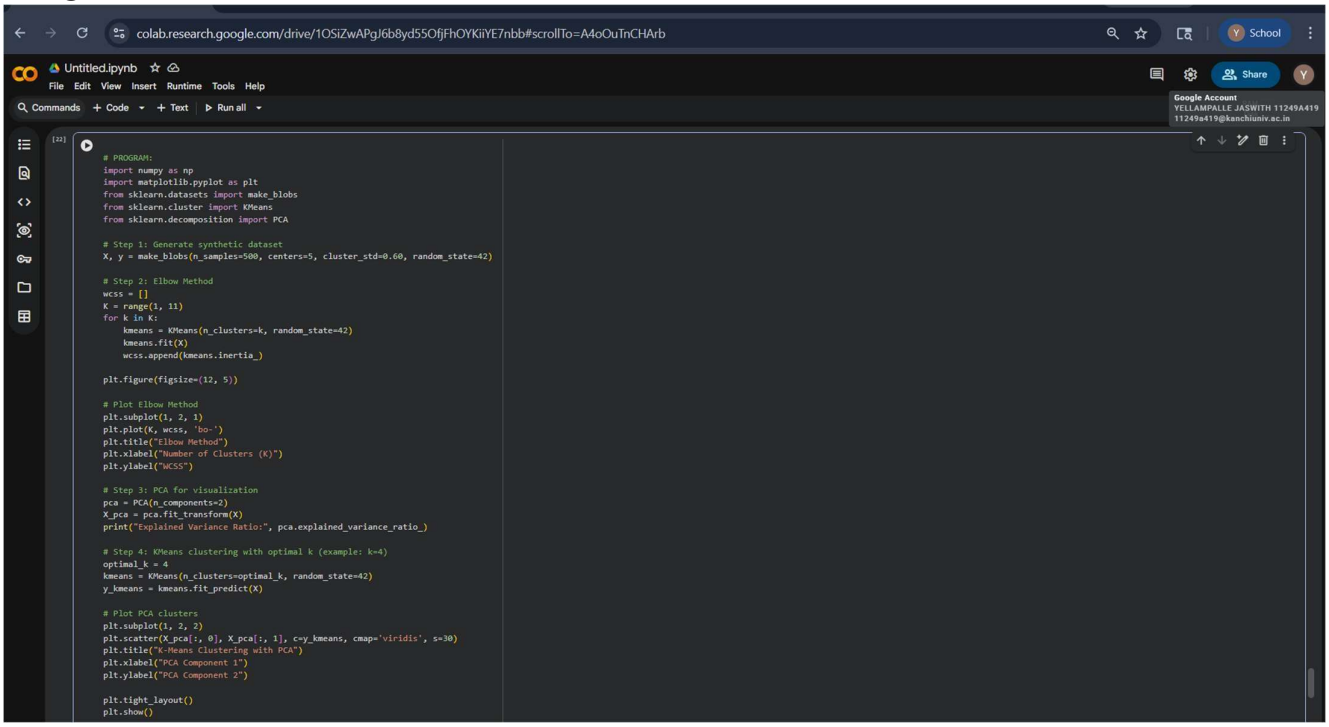
EXP-10

DATE-30-03-2026

Aim

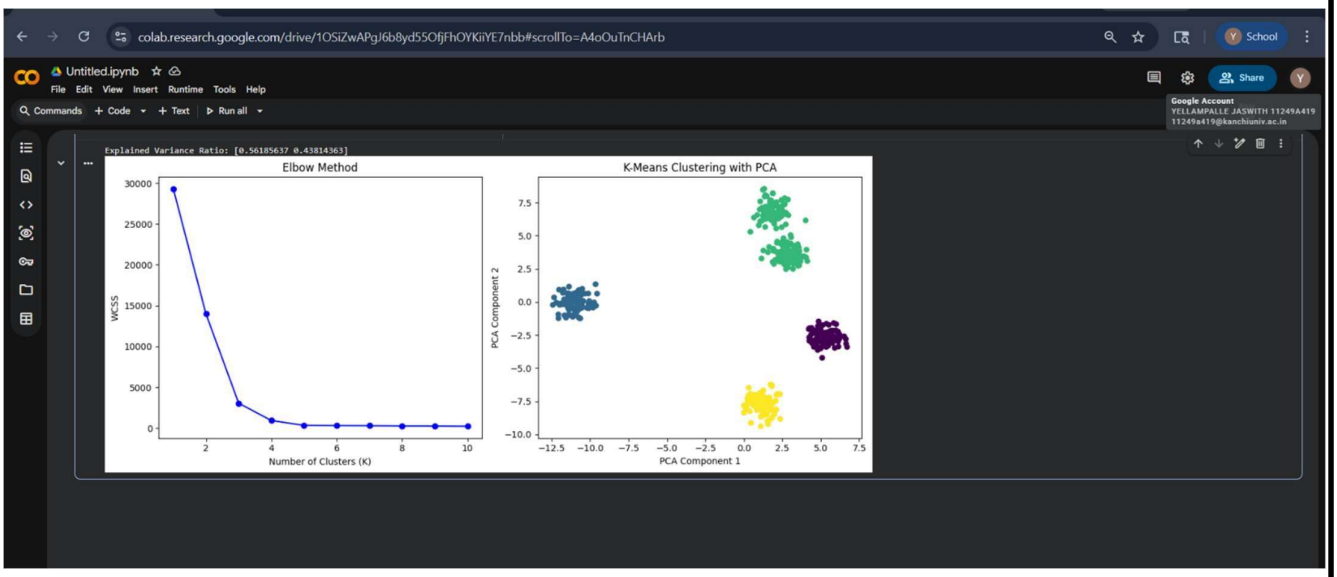
To perform K-Means clustering using Python along with PCA and the Elbow method.

Program:



```
# PROGRAM:
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Step 1: Generate synthetic dataset
X, y = make_blobs(n_samples=500, centers=5, cluster_std=0.60, random_state=42)

# Step 2: Elbow Method
wcss = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(12, 5))

# Plot Elbow Method
plt.subplot(1, 2, 1)
plt.plot(K, wcss, 'bo-')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")

# Step 3: PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Step 4: KMeans clustering with optimal k (example: k=4)
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Plot PCA clusters
plt.subplot(1, 2, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='viridis', s=30)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")

plt.tight_layout()
plt.show()
```

OUTPUT



Result

K-Means clustering was successfully implemented. The op mal number of clusters was iden fied using the Elbow method, and PCA was used to reduce dimensionality for be er visualiza on. The data was grouped into meaningful clusters effec vely.